

# Assignment 3 | Bridge Pattern

## Topic C — Reports: **Report** × **Formatter**

|                  |                                                                                                                         |          |                                 |
|------------------|-------------------------------------------------------------------------------------------------------------------------|----------|---------------------------------|
| Student          | Adilzhan Kuandykov                                                                                                      | Group    | SE-2526                         |
| Instructor       | Yerassyl Bekenov                                                                                                        | Language | Java, JDK 17 (JDK classes only) |
| Repository       | <a href="https://github.com/adilzhandev/design-pattern-bridge">https://github.com/adilzhandev/design-pattern-bridge</a> |          |                                 |
| Base commit      | 0da31fcda61f99ef99560021967baacf06fd89cf — I1/I2, checks T1-T5                                                          |          |                                 |
| Extension commit | e6ce14442ba2f9fa063fdf7d6062b117bf397f8a — I3, checks T6-T7                                                             |          |                                 |
| Submitted commit | head of main; the exact hash is in the Moodle submission text                                                           |          |                                 |

7/7

checks PASS

2 × 3

combinations, 0 combo subclasses

0

existing classes changed for I3

### Contents

1. Problem: two independent dimensions
2. Design: roles and UML class diagram
3. How a call crosses the bridge
4. Clean Code in five excerpts
5. Extension: adding MarkdownFormatter (I3)
6. Trade-off and comparison with Adapter
7. Test evidence (T1-T7)
8. References

## 1 Problem: two independent dimensions

A school report changes along two directions that do not depend on each other. Bridge gives each direction its own hierarchy and connects them with a single interface-typed reference, so either side can grow without touching the other.

### What is reported — Abstraction

**Report** → AttendanceReport (3 of 4 sessions = **75%**), GradeReport (avg of 70, 80, 90 = **80**). Owns the data and the calculation.

### How it is presented — Implementor

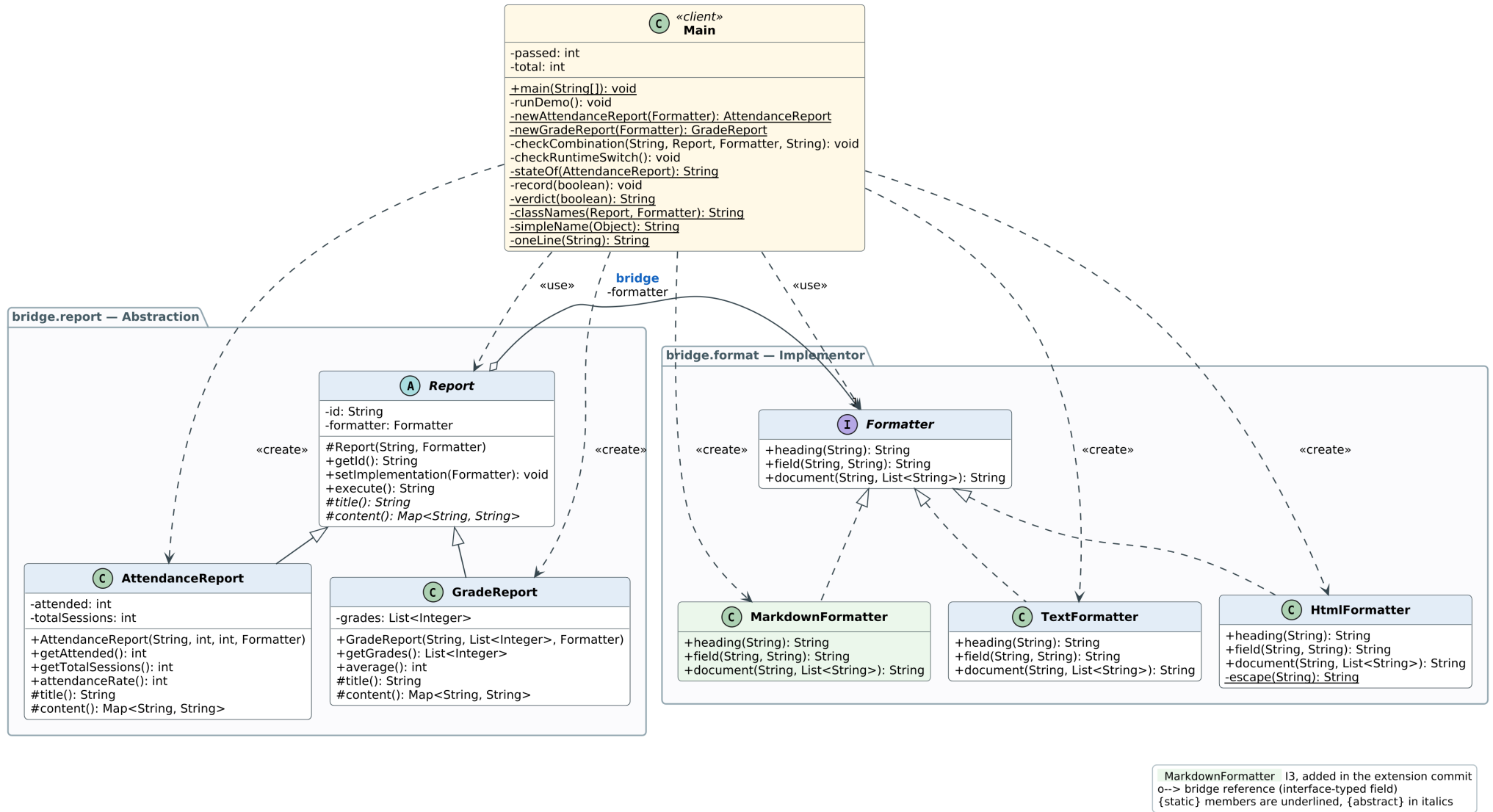
**Formatter** → TextFormatter, HtmlFormatter, MarkdownFormatter. Owns the low-level operations heading, field, document.

|                                              | Inheritance only                                     | Bridge (this project)                         |
|----------------------------------------------|------------------------------------------------------|-----------------------------------------------|
| <b>Classes for 2 reports × 3 formats</b>     | 6 combination subclasses (AttendanceTextReport, ...) | 2 reports + 3 formatters, combined at runtime |
| <b>Adding one format</b>                     | one new subclass per report                          | one new class, nothing else changes           |
| <b>Changing format of an existing object</b> | impossible (fixed by class)                          | setImplementation(Formatter)                  |

## 2 Design: roles and UML class diagram

| Role        | Class                                        | Responsibility                                              |
|-------------|----------------------------------------------|-------------------------------------------------------------|
| Abstraction | <b>Report</b> (abstract) · bridge.report     | stores id and the bridge field; execute() builds the report |
| A1 / A2     | <b>AttendanceReport / GradeReport</b>        | 3/4 → 75% / avg(70, 80, 90) → 80                            |
| Implementor | <b>Formatter</b> (interface) · bridge.format | low-level formatting contract                               |
| I1 / I2     | <b>TextFormatter / HtmlFormatter</b>         | plain text / escaped HTML                                   |
| I3          | <b>MarkdownFormatter</b>                     | Markdown, added in the extension commit                     |
| Client      | <b>Main</b> (default package)                | builds the combinations, runs T1-T7                         |

**Figure 1. UML class diagram of the final code (PlantUML)**



Report holds the interface-typed reference -formatter: Formatter (aggregation, labelled bridge). Main creates every concrete class and uses both abstractions. Static members are underlined, abstract ones in italics; I3 is highlighted green.

### 3 How a call crosses the bridge

**The bridge:** private Formatter formatter; in Report — typed by the interface, supplied through the constructor Report(String id, Formatter formatter) and replaceable with setImplementation(Formatter). The abstraction never creates, casts to or checks the type of a concrete formatter.

Report.execute() is the only workflow. It asks the subclass for its data with content() and delegates every formatting step — field, heading, document — to the stored Formatter (Figure 2).

In **T5** the client keeps one object and only re-points its formatter field. Identity is proven with current == original; the domain state (id=AR-1 attended=3/4 rate=75%) is compared before and after the switch, while the output changes from plain text to HTML.

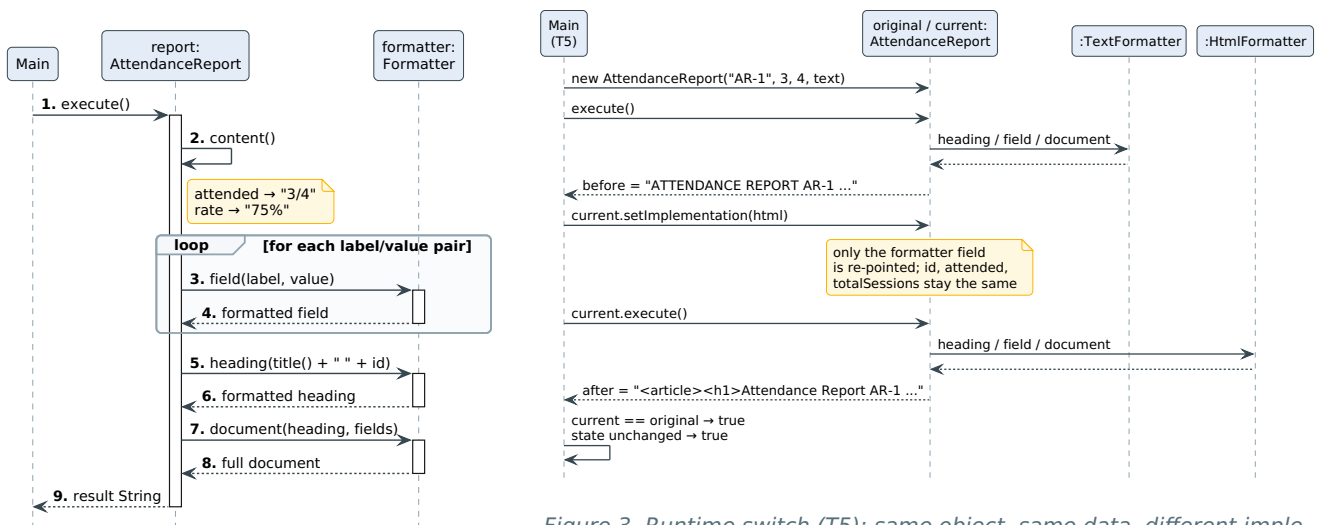


Figure 2. Delegation inside execute().

Figure 3. Runtime switch (T5): same object, same data, different implementation.

### 4 Clean Code in five excerpts

#### 1 Separation of responsibilities

```
AttendanceReport.content() · HtmlFormatter.field()

protected Map<String, String> content() {
    Map<String, String> content = new LinkedHashMap<>();
    content.put("attended", attended + "/" + totalSessions);
    content.put("rate", attendanceRate() + "%");
    return content;
}

public String field(String label, String value) {
    return "<li>" + escape(label) + ": " + escape(value) + "</li>";
}
```

**Benefit:** the report only calculates *what* to show; the formatter only decides *how* it looks. A new format never touches the percentage logic and a new report never touches HTML escaping — each class has one reason to change.

## 2 Meaningful names

Main.checkRuntimeSwitch() — T5

```
Report current = original;
current.setImplementation(second);
String after = current.execute();

boolean sameObject = current == original;
boolean stateUnchanged = stateBefore.equals(EXPECTED_STATE) && stateOf(original).equals(stateBefore);
```

**Benefit:** the check reads like its specification — *same object, state unchanged* — so it needs no comments. `attendanceRate()`, `setImplementation(Formatter)` and `newAttendanceReport(...)` follow the same rule.

## 3 Small, focused methods

HtmlFormatter.escape() · Main.verdict()

```
private static String escape(String text) {
    return text.replace("&", "&amp;")
               .replace("<", "&lt;")
               .replace(">", "&gt;");
}

private static String verdict(boolean pass) {
    return pass ? "PASS" : "FAIL";
}
```

**Benefit:** each method does one job and can be read and tested on its own; the escaping rule lives in exactly one place and is reused by `heading()` and `field()`.

## 4 No duplicated workflow or formatting logic

Report.execute() · Main.runDemo()

```
public String execute() {
    List<String> fields = new ArrayList<>();
    for (Map.Entry<String, String> entry : content().entrySet()) {
        fields.add(formatter.field(entry.getKey(), entry.getValue()));
    }
    return formatter.document(formatter.heading(title() + " " + id), fields);
}

checkCombination("T1", newAttendanceReport(text), text, EXPECTED_T1);
checkCombination("T2", newAttendanceReport(html), html, EXPECTED_T2);
checkCombination("T3", newGradeReport(text), text, EXPECTED_T3);
checkCombination("T4", newGradeReport(html), html, EXPECTED_T4);
```

**Benefit:** the “build a report” algorithm is written once for 2 reports × 3 formatters; subclasses only supply `title()` and `content()`. One `checkCombination` runs T1–T4, T6 and T7, so the PASS/FAIL logic and output format cannot drift apart.

## 5 Encapsulation of state

GradeReport – fields and constructor

```
private final List<Integer> grades;

public GradeReport(String id, List<Integer> grades, Formatter formatter) {
    super(id, formatter);
    if (grades.isEmpty()) {
        throw new IllegalArgumentException("grades must not be empty");
    }
    this.grades = List.copyOf(grades);
}
```

**Benefit:** data is private, final and validated; `List.copyOf` stores an unmodifiable copy. In Report the `id` is final and the private formatter changes only through `setImplementation`, which rejects null — exactly what T5 relies on.

## 5 Extension: adding MarkdownFormatter (I3)

The 2 × 2 solution with runtime switching was committed first (0da31fc, T1–T5 passing). I3 was then added in a separate commit (e6ce144). `extension.diff` was produced with `git diff 0da31fc HEAD -- src:`

| File in src/                             | Change   | Lines    |
|------------------------------------------|----------|----------|
| src/Main.java                            | modified | +8 / -0  |
| src/bridge/format/MarkdownFormatter.java | new file | +21 / -0 |

✓ Report,  
AttendanceReport,  
GradeReport unchanged

✓ Formatter interface  
unchanged

✓ TextFormatter,  
HtmlFormatter unchanged

This works because Report depends only on the Formatter contract. Main gained one import, two expected strings, one formatter instance and the T6/T7 calls; outside `src/` only `sources.txt` got one new line. T6 and T7 passed on the first run.

## 6 Trade-off and comparison with Adapter

**Trade-off.** The Formatter interface is a fixed contract. A new *format* costs one class, but a new kind of *formatting operation* — e.g. a `table(...)` with one row per student — forces a change to Formatter and to every implementor (I1, I2 and I3) at once. Bridge also adds indirection: one output is produced by two classes. For a single fixed format this would be over-engineering; here both dimensions are expected to grow.

|                        | Bridge                                                                     | Adapter                                                                                                                                                                                                                                                                    |
|------------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Intent</b>          | separate an abstraction from its implementation so both vary independently | make an existing class with an incompatible interface usable through the interface a client expects                                                                                                                                                                        |
| <b>When applied</b>    | designed up front                                                          | applied afterwards, to integrate existing code                                                                                                                                                                                                                             |
| <b>Structure</b>       | two hierarchies linked by composition                                      | one wrapper that converts calls to the adaptee                                                                                                                                                                                                                             |
| <b>In this project</b> | Report → Formatter: 2 reports × 3 formatters                               | a hypothetical PdfFormatterAdapter implements Formatter wrapping a third-party PdfWriter ( <code>writeTitle</code> , <code>writeParagraph</code> ) would plug into the bridge as one more implementor — its purpose is interface conversion, not decoupling two dimensions |

## 7 Test evidence (T1-T7)

Captured from `java -cp out Main --demo` (file `demo-output.txt`); every status is computed by comparing the actual result with the expected one. Line breaks inside results are shown as `\n`.

| Check | Participating classes                            | Status | Actual result                                                                                                                                                                                                                                         |
|-------|--------------------------------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T1    | AttendanceReport + TextFormatter                 | PASS   | ATTENDANCE REPORT AR-1\nattended: 3/4\nrate: 75%                                                                                                                                                                                                      |
| T2    | AttendanceReport + HtmlFormatter                 | PASS   | <article><h1>Attendance Report AR-1</h1><ul><li>attended: 3/4</li><li>rate: 75%</li></ul></article>                                                                                                                                                   |
| T3    | GradeReport + TextFormatter                      | PASS   | GRADE REPORT GR-1\ngrades: 70, 80, 90\naverage: 80                                                                                                                                                                                                    |
| T4    | GradeReport + HtmlFormatter                      | PASS   | <article><h1>Grade Report GR-1</h1><ul><li>grades: 70, 80, 90</li><li>average: 80</li></ul></article>                                                                                                                                                 |
| T5    | AttendanceReport: TextFormatter -> HtmlFormatter | PASS   | sameObject=true<br>stateUnchanged=true<br>state=id=AR-1 attended=3/4 rate=75%<br>before=ATTENDANCE REPORT AR-1\nattended: 3/4\nrate: 75%<br>after=<article><h1>Attendance Report AR-1</h1><ul><li>attended: 3/4</li><li>rate: 75%</li></ul></article> |
| T6    | AttendanceReport + MarkdownFormatter             | PASS   | # Attendance Report AR-1\n\n- **attended:** 3/4\n- **rate:** 75%                                                                                                                                                                                      |
| T7    | GradeReport + MarkdownFormatter                  | PASS   | # Grade Report GR-1\n\n- **grades:** 70, 80, 90\n- **average:** 80                                                                                                                                                                                    |

**SUMMARY: 7/7 PASS**

Build and run (from the project root)

```
javac --release 17 -encoding UTF-8 -d out "@sources.txt"
java -cp out Main --demo
```

## 8 References

1. E. Gamma, R. Helm, R. Johnson, J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994 — Adapter (pp. 139-150), Bridge (pp. 151-161).
2. A. Shvets. Refactoring.Guru — “Bridge”, “Adapter”. <https://refactoring.guru/design-patterns/bridge>, <https://refactoring.guru/design-patterns/adapter>
3. R. C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
4. Oracle. Java SE 17 API Specification (`java.util.List.copyOf`, `java.util.Objects.requireNonNull`). <https://docs.oracle.com/en/java/javase/17/docs/api/>
5. PlantUML Language Reference Guide — class and sequence diagrams. <https://plantuml.com>
6. Y. Bekenov. Assignment 3 | Bridge Pattern — instructions, ShP-2216, Astana IT University, 2026.